EvoDrive Lab: A Reproducible Procedural 2D Driving Benchmark for Neuroevolution and Lightweight Policy Optimization

Bishal Chhetri

Abstract

EvoDrive Lab is a Docker-first benchmark for studying learning and generalization in procedural 2D driving tasks. The benchmark combines deterministic track generation, a shared observation and action interface, and unified evaluation metrics with three training approaches: a custom genetic algorithm, NEAT, and a lightweight PPO-style baseline implemented directly in the repository. The central question is whether methods that achieve strong behavior on optimization-adjacent tracks generalize similarly to unseen procedural tracks. Across five benchmark seeds, NEAT achieved the strongest held-out test completion in the main comparison (0.879), closely followed by the custom genetic algorithm (0.866), while the lightweight PPO baseline failed to solve the task reliably (0.002). A seven-ray sensor configuration improved the genetic algorithm but did not improve NEAT, and longer procedural tracks substantially reduced held-out completion for both evolutionary methods. We position EvoDrive Lab as a small-scale, reproducible benchmark and open-source research artifact rather than a realistic autonomous-driving system, and we focus on completion, reward, crash rate, and episode length as the primary comparison metrics.

1 Introduction

Research code for driving agents often falls into one of two extremes: visually compelling demos with weak experimental structure, or benchmark-oriented systems that are difficult to run, inspect, and extend. Large platforms such as CARLA and MetaDrive have been influential for realism and scale, while procedural reinforcement-learning benchmarks such as Procgen and CoinRun have helped foreground generalization as a first-class evaluation target [3, 4, 1, 2]. EvoDrive Lab is intended to occupy a different point in that design space: a lightweight, Docker-first benchmark where multiple agents can be trained, replayed, and compared on procedurally generated 2D tracks without large infrastructure overhead.

The central question of this paper is whether different training methods generalize similarly to unseen procedural tracks. A method can appear effective during optimization yet still degrade on held-out environments. EvoDrive Lab makes that gap visible and measurable by pairing shared train, validation, and test suites with replay-oriented qualitative analysis. The benchmark is small enough to run on commodity hardware, but structured enough to support repeatable comparisons and paper-ready reporting.

This paper does not claim state-of-the-art autonomous driving performance. Instead, it contributes a compact, reproducible benchmark and an empirical comparison of a custom genetic algorithm, NEAT [6], and a lightweight PPO-style baseline inspired by proximal policy optimization [5]. The emphasis is on comparative behavior inside one controlled benchmark rather than broad conclusions about autonomous driving.

More concretely, the paper makes three contributions:

- a small procedural 2D driving benchmark that links quantitative evaluation with qualitative replay artifacts;
- a reproducible comparison of neuroevolution and lightweight policy optimization under a shared train/validation/test protocol;
- two compact ablations that test how sensor richness and track length affect generalization.

The remainder of the paper is organized as follows. Section 2 describes the benchmark setup and evaluation interface. Section 3 summarizes the compared training methods. Section 4 presents the experimental protocol. Section 5 reports the main comparison and ablations, and the final sections discuss limitations and conclusions.

2 Benchmark Setup

2.1 Task and Procedural Track Generation

EvoDrive Lab uses procedurally generated top-down driving tracks parameterized by a track seed and a small number of geometry controls. Each episode is defined by a centerline, left and right walls, a start gate, a finish gate, and intermediate checkpoint gates. The benchmark is designed to remain lightweight enough for laptop-scale experimentation while still exposing a meaningful train-versus-test generalization problem.

2.2 Observations and Actions

Each car observes the environment through forward-facing ray sensors and a small set of motion-state features. The default setting uses five rays in the main comparison, and the sensor ablation increases this to seven rays. Policies output continuous steering and throttle actions. Episodes terminate on crash or timeout.

2.3 Evaluation Splits and Metrics

The current benchmark uses fixed train, validation, and test suites, allowing the same policies to be evaluated both on optimization-adjacent tracks and on unseen held-out tracks.

The benchmark reports mean reward, mean completion, crash rate, and mean episode length. These values are currently the most stable and directly supported evaluation metrics in the codebase, and they form the basis of the quantitative analysis. Completion is the primary task-oriented metric because it captures progress toward solving the course, while reward, crash rate, and episode length provide complementary information about stability and failure modes. In addition to the primary metrics, the reporting pipeline derives a compact generalization-gap view by comparing validation performance with held-out test performance for the same trained policy.

3 Compared Methods

3.1 Genetic Algorithm

Genetic Algorithm. EvoDrive Lab includes a custom genetic algorithm that evolves policy weights generation by generation. In the frozen paper protocol, the GA runs for 12 iterations per seed and serves as the most natural fit for the live multi-car simulation because population turnover is directly visible in the interface and replay outputs. The GA therefore acts as both a competitive benchmark method and a useful qualitative reference for the visual side of the project.

3.2 NEAT

NEAT. The benchmark also integrates `neat-python` as a topology-evolving baseline based on NeuroEvolution of Augmenting Topologies [6]. It uses the same environment and evaluation suites as the other methods and also runs for 12 iterations per seed in the paper configuration. This allows comparison between direct parameter evolution and evolving network structure under a shared benchmark budget.

3.3 Lightweight PPO-style Baseline

PPO-style baseline. The repository contains a lightweight in-repo implementation of PPO-style policy optimization inspired by PPO [5]. In the paper configuration it runs for 8 outer iterations and 8192 total timesteps per seed. This baseline is intentionally modest: it is included to provide a policy-gradient comparison point, not to stand in for a large production RL stack such as a heavily tuned Stable-Baselines3-style pipeline.

3.4 Shared Comparison Protocol

All three methods share the same observation interface, action interface, train/validation/test suites, and evaluation metrics. The benchmark therefore isolates differences in training behavior rather than changing environment semantics between algorithms. This common protocol is important for interpretation: when one method underperforms, the explanation should be sought primarily in optimization behavior rather than in a mismatch of observation spaces, action spaces, or evaluation rules.

4 Experimental Protocol

4.1 Main Comparison

The main benchmark compares GA, NEAT, and the PPO-style baseline across five fixed random seeds: 7, 11, 23, 42, and 101. The benchmark uses deterministic train, validation, and test suites defined in code. For the main comparison, each method is trained under the frozen configuration committed in the repository and evaluated with the same reporting pipeline. In the main protocol, GA and NEAT each run for 12 iterations per seed, while the lightweight PPO baseline runs for 8 outer iterations with 8192 total timesteps.

Table 1: Main benchmark comparison on held-out test tracks.

Method	Test completion	Test reward	Crash rate	Val.-test gap
GA	0.866	-0.734	0.067	-0.066
NEAT	0.879	-0.787	0.000	0.001
PPO	0.002	-13.270	0.467	0.003

4.2 Ablations

The paper includes two compact ablations chosen for high value relative to implementation time. The sensor-count ablation compares five-ray and seven-ray observation settings. The track-length ablation compares a standard track profile with a longer profile defined by a higher segment count, slightly longer segment length, and slightly wider track geometry. These ablations were selected because they are already close to existing simulator controls and can be reproduced without introducing new algorithmic components or changing the overall learning problem.

4.3 Aggregation and Reporting

Each experiment family is aggregated across seeds into mean and standard-deviation summaries. The main paper tables and figures are generated directly from those outputs rather than from hand-edited spreadsheets. This design is intended to keep the reported numbers reproducible from the public repository state and to reduce the risk of transcription mistakes between code, plots, and manuscript tables.

The quantitative comparison focuses on four reported metrics:

- mean reward,
- mean completion,
- crash rate,
- mean episode length.

The analysis also uses validation-to-test generalization gaps as a compact way to describe whether a method that performs well on optimization-adjacent tracks maintains similar behavior on held-out procedural tracks. Negative gaps are possible in this setting because held-out tracks are sampled procedurally rather than ordered by explicit difficulty; a small negative gap therefore indicates that the held-out suite was not uniformly harder for that method.

5 Results

The paper reports validation and held-out test performance using completion, reward, crash rate, and mean episode length. The primary comparison focuses on whether the ranking of methods on validation tracks matches their behavior on unseen test tracks. In other words, the central question is not only which method performs best near the training distribution, but also which method degrades least when moved to held-out procedural tracks.

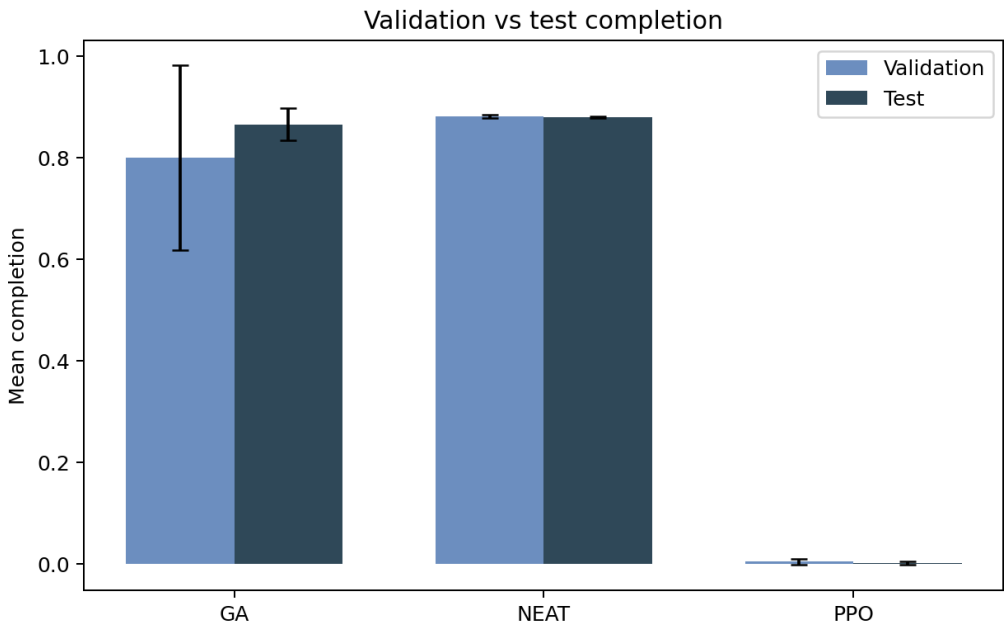

Figure 1: Validation versus test completion in the main benchmark.

5.1 Main Benchmark Comparison

Table 1 shows the main pattern of the benchmark. NEAT achieved the strongest held-out completion score, but only by a small margin over the custom genetic algorithm. The GA was notable because its mean test completion slightly exceeded its mean validation completion, suggesting that its learned behavior transferred well to the held-out seeds in this benchmark. NEAT, by contrast, showed almost no validation-to-test gap, indicating stable performance between the two suites. The lightweight PPO baseline did not learn a competitive driving policy under the current benchmark budget.

Figures 1–3 reinforce the same ranking from a few complementary views. Completion shows NEAT and GA clustered near one another and far ahead of PPO. Reward tells a similar story, although the reward ordering is less intuitive because the metric also reflects shaped penalties. Crash rate is especially revealing: NEAT achieved zero held-out crashes on average, GA remained low, and PPO crashed nearly half the time even though it often failed earlier through weak progress rather than outright course completion.

5.2 Sensor-count Ablation

The sensor ablation produced a method-dependent result. Increasing the observation count from five rays to seven improved GA held-out completion from 0.866 to 0.879 and improved PPO from 0.002 to 0.017, but NEAT declined slightly from 0.877 to 0.873. The most plausible interpretation is that additional sensing helped direct parameter evolution more than topology evolution in this compact task, while the PPO baseline still lacked enough optimization strength to capitalize consistently on the richer observation space.

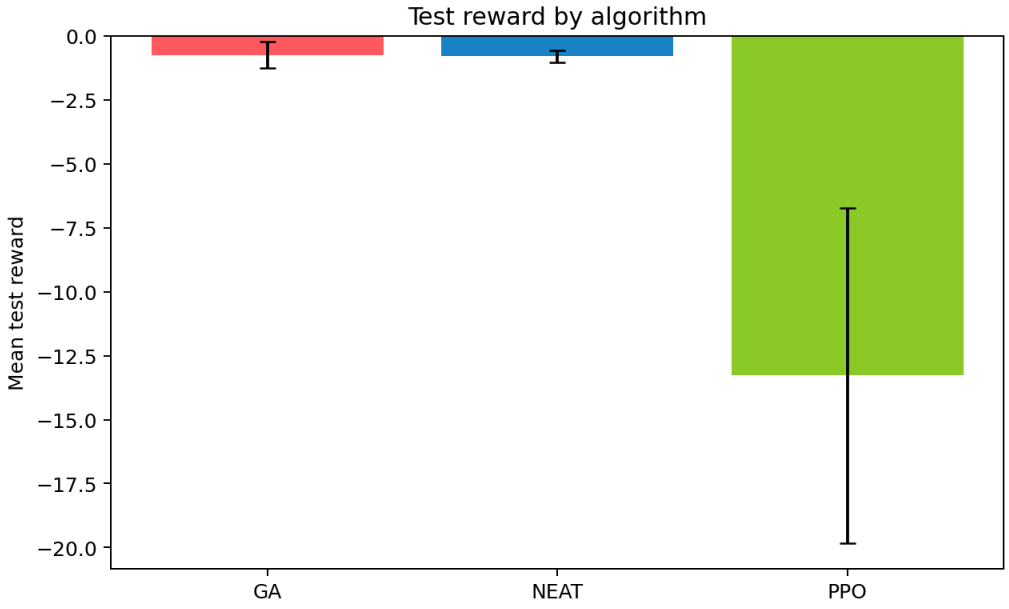

Figure 2: Held-out test reward by algorithm.

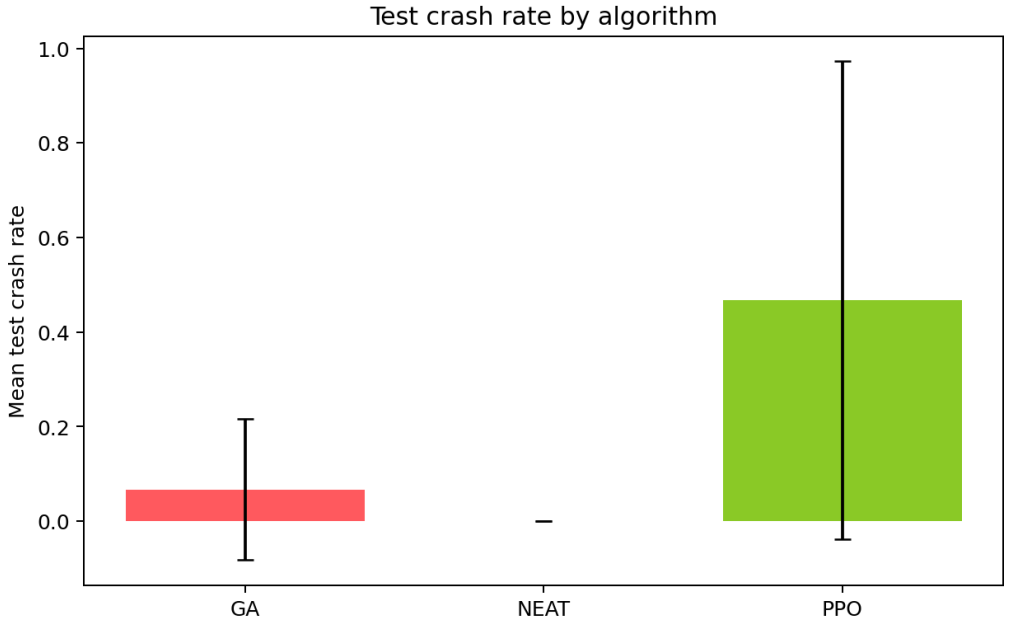

Figure 3: Held-out crash rate by algorithm.

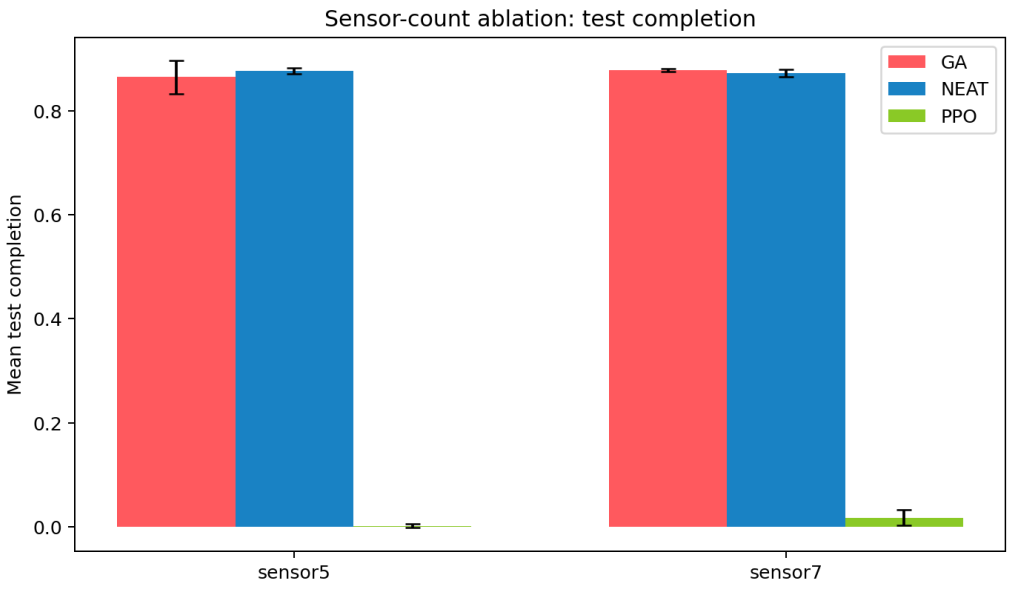

Figure 4: Held-out completion under the sensor-count ablation.

5.3 Track-length Ablation

The track-length ablation was more severe. Moving from the standard track profile to the longer profile reduced held-out completion for GA from 0.866 to 0.470 and for NEAT from 0.879 to 0.562. PPO remained near failure in both cases, though its completion increased slightly on the long-track condition from 0.002 to 0.007. This indicates that longer procedural tracks create a substantially harder generalization problem even for the stronger evolutionary methods and that benchmark geometry is a meaningful difficulty lever rather than a cosmetic variation.

5.4 Qualitative Replay Analysis

Figures 4 and 5 summarize the ablations visually, but the replay artifacts add an important qualitative check. For GA and NEAT, the median and best runs are visually close because both methods usually recover stable centerline-following behavior once training succeeds. PPO, on the other hand, never produced a qualitatively strong run in the selected cases: even its best replay showed negligible completion and poor reward. This difference matters because it shows that the failure of PPO in the current benchmark is not merely a statistical artifact of one metric, but also visible in the replay behavior itself.

6 Limitations

EvoDrive Lab is a simplified 2D benchmark and should not be interpreted as a realistic autonomous driving simulator. The benchmark scale is modest, the procedural suites are small, and the PPO baseline is intentionally lightweight rather than heavily tuned. The paper therefore focuses on comparative generalization trends inside this benchmark, not on real-world driving relevance or state-of-the-art claims.

The benchmark also omits several features that would matter in richer settings: traffic, perception noise, nontrivial vehicle dynamics, and broader hyperparameter sweeps. These omissions are

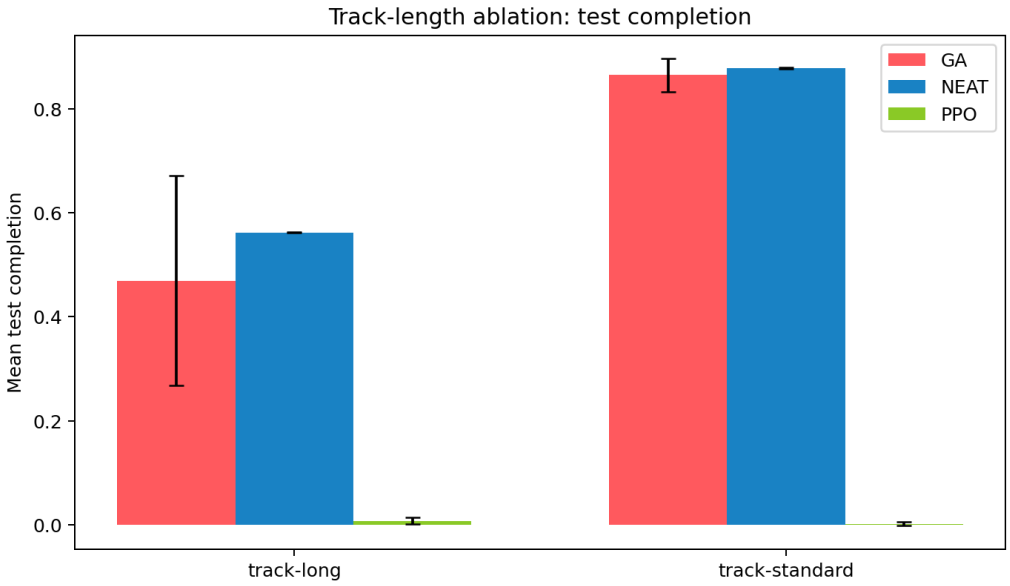

Figure 5: Held-out completion under the track-length ablation.

deliberate, but they narrow the scope of the conclusions. The value of EvoDrive Lab lies in reproducibility, visibility, and compact comparative experimentation, not in environmental realism.

7 Conclusion

EvoDrive Lab provides a compact benchmark for studying how different learning methods behave on unseen procedural 2D driving tracks. Its main contribution is not realism or scale, but a reproducible and inspectable comparison surface that connects benchmark metrics with visual replay evidence.

Across the completed benchmark, NEAT achieved the strongest held-out completion, the custom genetic algorithm remained highly competitive and benefited from a richer sensor configuration, and the lightweight PPO-style baseline failed to solve the task reliably under the current compute budget. Longer procedural tracks substantially reduced completion for both evolutionary methods, which suggests that track geometry is an important difficulty lever for future versions of the benchmark.

With a fixed experiment protocol, aggregate reporting pipeline, and open-source implementation, EvoDrive Lab can serve as a useful entry point for future work on lightweight driving benchmarks, neuroevolution, and procedural generalization. Natural next steps include stronger policy-gradient baselines, larger procedural suites, and richer qualitative analyses of failure recovery. The main empirical takeaway should still be framed narrowly: these results characterize comparative behavior inside this benchmark rather than broad claims about autonomous driving more generally.

References

[1] Karl Cobbe, Christopher Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. *arXiv preprint arXiv:1912.01588*, 2019.

[2] Karl Cobbe, Oleg Klimov, Christopher Hesse, Taehoon Kim, and John Schulman. Quantifying generalization in reinforcement learning. *arXiv preprint arXiv:1812.02341*, 2018.

[3] Alexey Dosovitskiy, German Ros, Felipe Codevilla, Antonio Lopez, and Vladlen Koltun. Carla: An open urban driving simulator. *arXiv preprint arXiv:1711.03938*, 2017.

[4] Quanyi Li, Zhenghao Peng, Lan Feng, Qihang Zhang, Zhenghai Xue, and Bolei Zhou. Metadrive: Composing diverse driving scenarios for generalizable reinforcement learning. *arXiv preprint arXiv:2109.12674*, 2021.

[5] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[6] Kenneth O. Stanley and Risto Miikkulainen. Evolving neural networks through augmenting topologies. *Evolutionary Computation*, 10(2):99–127, 2002.